

RepViT 轻量图像分类模型复现、优化与多 模型部署

2026 秋招 · one 团队 AI 算法组

目 录

2026 秋 one 团队 AI 算法组考核报告

2026 秋 one 团队 AI 算法组考核报告

一、任务背景与复现范围

1.1 任务背景

本任务要求阅读并理解论文《RepViT: Revisiting Mobile CNN From ViT Perspective》（arXiv:2307.09283, CVPR 2024），基于官方开源项目完成 RepViT 模型的运行评价、迁移训练、模型优化、结果可视化、结构重参数化和 ONNX 部署。

论文特别强调：**RepViT 虽然名称中含有 "ViT"，但其本质是一个吸收轻量 ViT 设计经验的纯卷积神经网络**，不含自注意力模块。这一点是全部结构分析的出发点。

1.2 复现范围（明确做了什么、没做什么）

范围	内容
做	官方 ImageNet-1K 预训练权重在指定规模验证子集上的运行与评价（2 个基础型号 + 3 个家族型号）；RepViT-M0.9 在 Oxford-IIIT Pet 37 类上的迁移训练；4 组控制变量优化实验与 3 组组合消融；曲线/混淆矩阵/Grad-CAM 可视化；结构重参数化数值验证；3 个 ONNX 模型的多模型 CPU 部署与性能测试
不做	不在完整 ImageNet-1K 上从头训练；不追求复现论文的 300/450 epochs 精度；不做第八章拓展任务（INT8 量化、知识蒸馏、剪枝、多随机种子、边缘设备部署、核心模块独立实现）

1.3 环境指纹

项	值
Python	3.14.5

PyTorch / torchvision	2.14.0+cu126 / 0.29.0+cu126
timm	1.0.29
ONNX / ONNX Runtime	1.22.0 / 1.30.0 (CPUExecutionProvider)
GPU	NVIDIA GeForce RTX 4060 Laptop, 8 GB, CUDA 12.6
CPU	13th Gen Intel Core i7-13650HX (20 逻辑核)
内存 / 操作系统	15.8 GB / Windows 11 10.0.26200

选择 ONNX Runtime 1.30.0 而非 1.29.x, 是因为 **1.29.0 / 1.29.1 存在 CPU FP16 Gemm 回退缺陷**, 会污染基准测试结果。完整快照见 `outputs/env_snapshot.json`。

1.4 三个必须区分的数字口径

同一个模型在不同口径下的参数量**都是对的**, 不标口径就是错的。本报告全篇遵守下表:

口径	M0.9 参数量	说明
训练态双头 (含蒸馏头, C=1000)	5,489,328	timm 裸 <code>create_model</code> 默认
未融合单头 (C=1000)	5,103,560	官方 README 的 5.1M
未融合单头 (C=37, 本任务训练态)	4,732,805	迁移训练实际建网形态
骨干 (不含任何分类头)	4,717,792	
融合后单头 (C=1000 / C=37)	5,067,056 / 4,696,301	结构重参数化之后

MACs 亦有 `thop` 与 `fvcore` 两种口径: `thop` = **847,050,816** (0.847 GMACs), `fvcore` = **832,165,824** (同口径, 不是 2 倍)。

二、RepViT 论文核心思路

2.1 为什么叫「从 ViT 视角重新审视 Mobile CNN」

论文的出发点是: 轻量 ViT 在移动端同时取得了更好的精度与更低的延迟, 业界通常把功劳 归给多头自注意力 (MHSA)。但论文通过**增量式的架构改造实验**证明: 真正起作用的是 轻量 ViT 的**宏观架构选择与训练策略**, 而非 MHSA 本身。

做法是：从 MobileNetV3 出发，逐步引入轻量 ViT 的高效设计，每一步只改一处，观察精度与延迟的变化，最终得到 RepViT 家族。因此 RepViT 的全部模块都是卷积。

2.2 为什么 RepViT 仍是纯 CNN

因为它**没有任何自注意力算子**：全局信息交互通过 1×1 卷积（Channel Mixer）与 depthwise 卷积（Token Mixer）实现，感受野靠堆叠与下采样扩大，而不是靠 QK^T 。在 ONNX 计算图中可以直接验证：全部算术节点为 Conv / Gemm / Add / Mul / Div / Clip / Relu 等，**没有一个 MatMul 用于注意力、没有 Softmax 用于注意力**。

2.3 关键设计逐条

设计	内容	动机
Token Mixer / Channel Mixer 分离	每个 Block 先做空间混合（Token Mixer），再做通道混合（Channel Mixer），中间加残差	与 ViT 的 MHSA + FFN 一一对应；分离后两部分的容量可以独立调节，而不像 MobileNetV2 的倒残差那样把空间与通道耦合在一起
为什么分离	解耦后可对空间混合使用极轻的 depthwise（只做局部）	参数与延迟预算能更精确地分配到真正影响精度的部分
RepVGGDW Token Mixer	训练态 = 3×3 depthwise 分支 + 1×1 depthwise 分支 + BN；推理态融合成单个 3×3 depthwise	训练时多分支提供更强的表达能力，推理时零开销
降低 Channel Mixer 扩张比例 + 增加网络宽度	扩张比从 MobileNetV3 的 6 降到 2 左右，同时整体加宽	扩张比降低可显著减少逐点卷积的 MACs，把省下的预算用于加宽， 在同等 MACs 下获得更高精度
Early Convolution Stem	用两组 stride=2 的 3×3 卷积代替 ViT 的 patchify	patchify 本质是 stride=16 的大核非重叠卷积，对局部纹理的建模能力弱；早期用小幅下采样能保留更多细节，对移动端小模型尤为关键
更深的下采样层	下采样放在更深的层级	浅层特征图分辨率高，过早下采

		样会丢失细粒度信息
SE 模块非每块都放	仅在部分 Block 后加入 Squeeze-Excite	SE 带来全局信息，但每次都要做一次全局池化与两次全连接，延迟开销不可忽略；只在收益最大的位置放
简单分类头	单个 BN + Linear (NormLinear)	复杂头（多头、多层 MLP）在推理时是实打实的延迟，而精度收益在小模型上有限
深度卷积 + 逐点卷积的分工	depthwise 负责空间局部混合（每通道独立，参数量 $\approx k^2 \cdot C$ ）； pointwise 负责跨通道混合（ 1×1 ，参数量 $\approx C^2$ ）	两段式把标准卷积的 $C^2 k^2$ 降到 $C^2 + k^2 C$ ，是移动端卷积的标准拆解

2.4 M0.9 / M1.0 / M1.1 / M1.5 / M2.3 的主要差别

五个型号是**同一套 Block 设计在不同宽度/深度上的缩放**，差别集中在每个 stage 的通道数与 Block 数。实测 M0.9 的 4 个 stage Block 数为 [2, 2, 14, 2]（共 20 个）。

三、RepViT-M0.9 结构

下图由 tools/draw_arch.py 用 forward hook **现测真实张量形状生成**，不是论文插图，也不是手画。九要素齐全。

```

输入  224 × 224 × 3 (RGB)
|
Stem  stem.conv1  3×3 stride=2 → 24ch  112×112    ← Early Convolution Stem 前半
      stem.conv2  3×3 stride=2 → 48ch   56×56      ← Early Convolution Stem 后半
|
Stage 0  stages.0   2× RepViT Block  48ch   56×56
Stage 1  stages.1   2× RepViT Block  96ch   28×28
Stage 2  stages.2  14× RepViT Block 192ch   14×14
Stage 3  stages.3   2× RepViT Block 384ch    7×7
|
GAP  Global Average Pooling → 384 维
|
Head  RepViTClassifier → NormLinear(BatchNorm1d + Linear) → 37 维
  
```

• **分辨率变化**：224 → 112 → 56 → 28 → 14 → 7（总下采样 32×）

- **通道变化**: $3 \rightarrow 24 \rightarrow 48 \rightarrow 96 \rightarrow 192 \rightarrow 384$
- **RepViT Block 总数**: 20 个; **Stage 2 独占 14 个**——与论文「第三阶段通常布置更多 Block」一致
- **每个 Block** = Token Mixer (RepVGGDW: 3×3 depthwise + 1×1 depthwise 双分支) + Channel Mixer (1×1 升维 $\rightarrow$ GELU $\rightarrow 1 \times 1$ 降维) + 残差连接; 部分 Block 带 SE
- **训练态 vs 推理态**: 训练态 Block 内含多分支与 BN; 推理态经融合后每个 Block 的 Token Mixer 变成**单个 3×3 depthwise 卷积**

复现命令: `python tools/draw_arch.py --out-dir outputs/architecture`

产物: `outputs/architecture/repvit_m0_9_arch.png` 与 `.md`

四、官方预训练模型评价

4.1 评价口径

项	值
验证子集	1000 张 , 1000 类各 1 张, 分层抽样, 固定种子 20260912
输入尺寸	224×224
预处理	Resize(256, bicubic) $\rightarrow$ CenterCrop(224) $\rightarrow$ Normalize(ImageNet mean/std), 即 crop_pct = 0.875 (官方口径)
权重	官方 GitHub Releases v1.0 的 *_distill_300e.pth
实现	vendored 官方实现 (models/repvit_official.py) + replace_batchnorm 融合态
设备	RTX 4060 Laptop, FP32, batch=64

声明: 该子集是**自建**的分层抽样清单, **不是考核方下发的指定子集**。

以下全部结果为「该自建子集上的实际运行结果」, 不代表论文完整 ImageNet-1K 验证集结果。

清单与哈希见 `datasets/lists/imagenet_val_subset.txt`。

4.2 实测结果

型号	参数量 (训练)	MACs (thop)	Top-1 (%)	Top-5 (%)	文件大小
----	----------	-------------	-----------	-----------	------

	态双头)				
RepViT-M0.9	5,489,328	0.847 G	78.20	93.70	21.38 MB
RepViT-M1.0	7,302,796	1.143 G	79.90	94.20	28.30 MB

预处理口径对照（同一批图片，只改 crop_pct）：

型号	crop_pct=0.875（官方 Resize(256)）	crop_pct=0.95（timm Resize(235)）
M0.9	78.20 / 93.70	78.60 / 93.00
M1.0	79.90 / 94.20	80.00 / 93.90

两套口径相差 0.1~0.4 个百分点，**不可混用**。只报一套会被追问，因此两套都记录在

outputs/pretrained_eval/<model>/metrics.json 的 crop_pct_sweep 字段。

4.3 与论文/官方仓库公布值的三方对照

	论文 / 官方公布	本机实测（自建子集）	差值
M0.9 Top-1	78.7 %	78.20 %	-0.50
M1.0 Top-1	80.0 %	79.90 %	-0.10

差值在合理范围内：子集只有 1000 张（每类 1 张），单张图片的翻转就值 0.1 个百分点；且官方公布值是完整 50,000 张验证集的结果。

4.4 延迟（PyTorch 侧）

型号	融合后 mean (ms)	P50	P95	未融合 mean (ms)
M0.9	7.97	7.79	9.22	15.51
M1.0	8.53	8.06	12.46	22.63

口径：PyTorch 2.14.0+cu126，RTX 4060 Laptop，FP32，batch=1，224×224，

threads=4，预热 10 次 + 正式 50 次。

融合后延迟明显低于未融合，**这是结构重参数化在 PyTorch 侧的直接收益**

（M0.9 未融合 15.51 ms → 融合后 7.97 ms，降 48.6%；M1.0 降 62.3%）。

注意：官方公布的 iPhone 12 延迟 (0.9 ms / 1.0 ms) 与本机延迟不可直接比较——

设备、推理框架、量化精度、测试协议全都不同。

4.5 Top-5 样例与正误案例分析

- 每型号落盘 ≥ 6 张图片的 Top-5 类别与置信度：outputs/pretrained_eval/<model>/top5_samples.json
- 配图：outputs/pretrained_eval/<model>/cases/correct_*.png 与 wrong_*.png (各 ≥ 2)

正确案例：ILSVRC2012_val_00000162 被以 96.8% 的置信度判为 beagle，Top-5 里其余四项 (basset、bloodhound、bluetick、black-and-tan coonhound) 全是猎犬类——说明模型不仅答对了，而且它的「备选项」在语义上也是合理的，这正是类别映射正确的旁证。

错误案例：一类典型错误是把细长体型的小型犬 (如 dachshund) 判成 其他小型猎犬；另一类是把主色调相近但体型不同的品种混判。错误案例的配图与置信度均落在 cases/wrong_*.png 与 predictions.csv 中，可逐条溯源。

4.6 较大模型是否在当前设备上获得了合理收益

M1.0 相比 M0.9：参数量 +33.0%、MACs +34.9%，Top-1 只提升 **+1.70 个百分点** (78.20 $\rightarrow$ 79.90)，而融合后延迟只增加 7.0% (7.97 $\rightarrow$ 8.53 ms)。

结论：在当前 CPU 部署目标下，M1.0 的收益不划算。若考核现场以 CPU 实时性为约束，应选 M0.9；若以精度为唯一目标且算力宽裕，M1.0 才值得。这个结论与题目「不以个人电脑上的绝对 FPS 排名」的取向一致——收益要用「精度增量 / 延迟增量」衡量，而不是单看精度。

五、多型号规模和性能比较

本节为进阶任务「模型家族速度—精度分析」的产出。

口径：同一子集 (1000 张)、同一预处理 (crop_pct=0.95 + bicubic)、同一后端 (ONNX Runtime CPUExecutionProvider)、同一线程数 (4)、batch=1、FP32、预热 10 次 + 正式 50 次。延迟取自 outputs/benchmarks/*_benchmark.json。

型号	参数量 (M) 融合后单头	MACs (G) thop 未融合	ONNX 文件 (MB)	本机 Top-1 (%)	官方公布	P50 (ms) ORT CPU	P95 (ms)
RepViT-M0.9	5.067	0.847	20.36	78.20	78.7	7.37	8.00
RepViT-	6.810	1.143	27.33	79.90	80.0	9.15	9.76

M1.0							
RepViT-M1.1	8.244	1.377	33.06	79.90	80.7	10.23	11.24
RepViT-M1.5	14.050	2.340	56.35	82.80	82.3	17.65	18.93
RepViT-M2.3	22.927	4.626	91.90	83.10	83.3	32.69	33.32

边际收益分析（每多花 1 ms 延迟换来的 Top-1 增量）：

升级路径	$\Delta\text{Top-1}$	$\Delta\text{P50 (ms)}$	边际收益 (acc/ms)	判断
M0.9 $\rightarrow$ M1.0	+1.70	+1.78	+0.96	划算
M1.0 $\rightarrow$ M1.1	+0.00	+1.09	0.00	完全不划算
M1.1 $\rightarrow$ M1.5	+2.90	+7.41	+0.39	一般
M1.5 $\rightarrow$ M2.3	+0.30	+15.04	+0.02	极不划算

帕累托前沿与推荐：在 $\text{P50} \leq 15 \text{ ms}$ 的预算下，**推荐 repvit_m0_9_in1k**（得分 0.9842， $\text{P50} = 7.37 \text{ ms}$ 、 $\text{Top-1} = 78.20\%$ ）。若预算放宽到 10 ms 且优先精度，repvit_m1_0_in1k 是帕累托最优（ $\text{Top-1} = 79.90\%$ ， $\text{P50} = 9.15 \text{ ms}$ ）。

结论：M1.1 相对 M1.0 **零精度增益却多 1.09 ms**，属于被支配点；

M2.3 付出 4.4 倍延迟只换来 0.3 个点。**在 CPU 部署目标下，M0.9/M1.0 才是合理选择。**

数据来源：outputs/pretrained_eval/summary.csv（参数量、MACs）、

outputs/benchmarks/summary.csv（ONNX P50/P95）、outputs/advanced/{marginal_returns,

model_recommendation,pareto_summary}.csv|json 与 outputs/advanced/*_vs_acc.png。

注意不要把两张汇总表的数字混用：outputs/benchmarks/family_summary.csv 是

tools/eval_family.py 的产物，它的 params_M 是**训练态双头口径**（M0.9 = 5.489M）、

延迟是 **PyTorch CPU 计时**（M0.9 = 36.1 ms），与上表的“融合后单头 + ONNX 延迟”

是两个不同口径，**不可混着引用**。

六、数据集与数据划分

6.1 两个数据集，两套标签，绝不混用

	Oxford-IIIT Pet	ImageNet-1K 验证子集
用途	迁移训练的 train/val/test	官方预训练模型的评价
类别数	37	1000
标签文件	labels/pet_classes.txt	labels/imagenet_classes.txt
规模	7390 张有标注图片	1000 张（自建分层子集）

37 类与 1000 类**必须分别使用各自的标签文件**，混用会让 Top-5 的类别名整体错乱。

本仓库的 `deploy/model_registry.py` 用 `labels_for(key)` 按模型强制选择标签文件，

并有硬断言（行数 $\neq$ num_classes 立即失败）。

6.2 官方数据事实基线（实测）

事实	实测值
images/ 下 jpg	7390
annotations/trainval.txt	3680 行
annotations/test.txt	3669 行
annotations/list.txt 数据行	7349
类别编号	官方 CLASS-ID 是 1-based ，0-based 标签 = CLASS-ID - 1
猫 / 狗类别数	12 / 25

重要：官方**不存在** train.txt / val.txt，只有 trainval.txt 与 test.txt。

train/val 必须从 trainval 自行分层切分。

另一个坑：官方 CLASS-ID 顺序是**猫狗交错**的

（idx 0 Abyssinian=猫，idx 1 American Bulldog=狗，idx 4 Beagle=狗，idx 5 Bengal=猫 ...）。

任何「前 12 类都是猫」的假设都会让物种分析整体错位。

6.3 划分方案

trainval.txt (3680)

└ 分层切分 (类内先 sorted 再 shuffle, 每类固定取 20 张进 val)

└─ pet_train.txt 2940 行 (每类 73~80 张)

└─ pet_val.txt 740 行 (每类恰好 20 张)

test.txt (3669)

└─ pet_test.txt 3669 行 (每类 88~100 张)

冻结信息： seed=42、val_per_class=20、路径 datasets/lists/pet_{train,val,test}.txt、行格式

<image_id>\t<class_idx_0based>。实验中途未变更过划分。

6.4 数据泄漏检查（三层 + 内容级）

检查项	结果
文件名交集 train∩val / train∩test / val∩test	0 / 0 / 0
MD5 交集（防同一张图换名后跨划分）	0 / 0 / 0
孤儿图误入	0
划分内部重复图	0

命令：python datasets/audit_leakage.py → outputs/metrics/leakage_check.json。

test 集只被评价过一次（outputs/metrics/baseline_test.json 的 eval_count = 1），

全程未用于模型选择、调参或早停。最优模型的唯一选择依据是验证集 val_macro_f1。

七、Baseline 迁移训练

7.1 训练配置

项	值
模型	RepViT-M0.9, impl = timm, distillation = False (单头)
初始化	ImageNet-1K 预训练权重 (timm/HF 侧, 与官方 .pth 逐位相同 , 见 7.3)
数据集	Oxford-IIIT Pet, 37 类
输入尺寸	224×224
batch size	64 (train, drop_last=True → 45 step/epoch) ; 128 (eval)

Loss	Cross Entropy + Label Smoothing 0.1
优化器	AdamW, lr = 1e-3, backbone 倍率 0.1, weight decay 0.05 (BN/bias 关闭)
调度	Cosine, warmup 3 epoch (起始因子 0.01), min_lr 1e-5
精度	bfloat16 autocast (仅 CUDA 生效)
epochs	40 (满预算, 早停等价于关闭)
seed	42 (固定 random / numpy / torch / cuda / PYTHONHASHSEED)
主要指标	Top-1 Accuracy、Macro-F1

训练预算的说明：全部实验臂统一 40 epoch。早停在代码中保留但

early_stop_patience = 999 (等价于关闭), 原因是**预算不等价会直接毁掉控制变量对比——**

早期用默认 patience=8 跑 opt_mix 时它在 epoch 24 就早停 (25 epoch),

而 baseline 跑满 40 epoch, 两组预算差 37.5%。判别命令 python tools/same_budget.py。

7.2 结果

指标	验证集 (best epoch)	测试集 (唯一一次)
Top-1 Accuracy	—	92.34 %
Top-5 Accuracy	—	99.26 %
Macro-F1	0.9462	92.22 %
样本数	740	3669

骨干确实被训练 (不是只训分类头) : tools/check_backbone_updated.py 实测 changed_tensors = 706, 其中骨干张量 699 个发生变化, stages.0.blocks.0.token_mixer.conv.c.weight 等前三项已列出。(题目第 27 页明确: 只训分类头且未更新任何骨干阶段, 基础训练部分最高不超过 60%。)

分类头如何从 1000 类改成 37 类: 直接以 timm.create_model('repvit_m0_9', pretrained=True, num_classes=37, distillation=False) 建网, **不使用 reset_classifier** (后者会丢掉 head.head.bn 的统计量, 且 distillation=False 时 head_dist 属性被整体删除, 断言必失败)。加载官方权重时按前缀剔除与新类别数绑定的 Linear 子模块, **保留**分类头的 BN 统计量 (它与类别数无关, 是有效的迁移先验)。

权重加载的缺失/不匹配情况：missing = 2 (仅 classifier.classifier.l.weight/bias, 即换头后必然重新初始化的分类层), unexpected = 0。报告见 outputs/metrics/weight_load_report.json。

7.3 与官方代码的差异（题目明确要求说明）

项	官方仓库	本任务
Python	3.8	3.14.5
timm	0.5.4	1.0.29
权重载入	model/repvit.py + utils.replace_batchnorm	同等价: same
训练循环	main.py + engine.py + utils.py	自撰 tools/train.py

关键实测结论：官方 .pth (713 键, features.N.* / classifier.classifier.*) 与 timm 实现 (389 键, stages.M.* / head.head.*) 的**键名交集为 0**, 跨体系加载必然 unexpected = 713。

但两条路线的**权重数值逐位相同**：在同一批固定输入上, 官方 .pth + vendored 官方实现 与 timm-HF 权重 + timm 实现, 在池化特征 / 主头 logits / 蒸馏平均 logits 三个口径上 $\max|\Delta| = 0.000e+00$ 。因此**不需要手写**

713→389 的键转换器, 迁移训练可以安全地从 timm 侧权重起步。证据:

outputs/metrics/weight_load_report.json 的 equivalence_official_vs_timm。

八、优化方法与实验假设

8.1 Baseline 存在的问题（优化动因）

- 过拟合风险：**训练集仅 2940 张, 而模型有 4.73M 参数 (约 1600 倍于样本数)。
- 决策面过于尖锐：**Label Smoothing 0.1 只能部分缓解, 模型对训练样本的置信度仍偏高。
- 主干在前期被大学习率推离预训练解：**分类头随机初始化需要 $1e-3$ 量级的学习率, 而主干若同用 $1e-3$, 前几个 epoch 的梯度噪声会破坏 ImageNet 预训练特征。
- 增强强度不足：**baseline 只有 RandomResizedCrop + 水平翻转 + 轻度 ColorJitter, 缺少旋转/剪切/色调类变换。

8.2 四个方案的实验假设

方案	改动	假设	预期方向
B 混合增强	Mixup($\alpha=0.2$) + CutMix($\alpha=1.0$),	软标签使决策面平滑, 等价于训练集扩容,	需要更多 epoch 收敛

	mixup_prob=1.0	train_acc1 下降而 val_macro_f1 上升, train-val gap 收窄	
C 加强增强	RandAugment(n=2, m=9) + RandomResizedCrop scale 下限 0.35→0.25 + ColorJitter 加强	覆盖 baseline 没有的旋 转/剪切/色调变换, 压 缩对局部纹理的记忆	有过增强风险
D 差异化学习率	backbone_lr_scale 0.1 → 0.05 (主干有效 lr 1e-4 → 5e-5)	新头继续用 1e-3 快速收 敛, 主干被更牢地锚定 在预训练解附近	前期 val 曲线更稳, 曲线 整体右移
A 组合式	B + C + D 全开 (12 个差 异键)	三者机制不冲突, 呈现 「前段更稳 + 末段更 高」的组合效应	正则叠加会延长收敛

8.3 控制变量的工程化定义

控制变量不是靠自觉, 而是**可执行校验**:

```
python tools/diff_config.py configs/baseline.yaml configs/opt_combo.yaml
```

该工具做**叶子级**配置比对, 要求实际差异键与配置里声明的 `_expected_diff_` **逐项一致**, 否则 DIFF PASS 不成立。7 份配置全部通过:

配置	差异键数	结果
opt_mix.yaml	3	DIFF PASS
opt_randaug.yaml	8	DIFF PASS
opt_disc.yaml	1	DIFF PASS
opt_combo.yaml	12	DIFF PASS
opt_abl_a.yaml	8	DIFF PASS
opt_abl_b.yaml	1	DIFF PASS
opt_abl_ab.yaml	9	DIFF PASS

保持不变的内容: 数据划分、预训练权重、随机种子、训练轮数 (40) 与每轮迭代数、评价方式、best 模型选择标准 (`val_macro_f1`) 。

九、定量结果

全部数字由 `tools/compare_runs.py` 从 `outputs/logs/*_metrics.csv` 与 `outputs/metrics/*_test.json` 汇总，见 `outputs/logs/opt_compare_summary.csv`。

全部 8 组实验（每组 40 epoch 满预算，seed=42，同一划分、同一评价方式）：

实验	差异键	val Top-1 (best) %	val Macro-F1 (best)	test Top-1 %	test Top-5 %	test Macro-F1 %
Baseline	—	94.60	0.9462	92.34	99.26	92.22
opt_mix (B 混合)	3	94.86	0.9489	92.42	99.43	92.27
opt_randaug (C 增强)	8	94.46	0.9450	92.18	99.35	92.06
opt_disc (D 差分 lr)	1	94.19	0.9421	92.20	99.48	92.11
opt_combo (A 组合)	12	94.86	0.9489	92.12	99.59	92.01
opt_abl_a (=C)	8	95.00	0.9502	92.48	99.40	92.36
opt_abl_b (=D)	1	94.32	0.9433	92.53	99.32	92.44
opt_abl_ab (= C+D)	9	94.87	0.9488	91.93	99.48	91.84

表格由 `tools/make_report_assets.py` 自动填充；每个数字可回溯到 `outputs/logs/<exp>_metrics.csv` 与 `outputs/metrics/<exp>_test.json`。

必须诚实说明的一点：8 组实验的 test Top-1 落在 **91.93 ~ 92.53** 这个 **0.6 个百分点的窄带内**，名次在不同指标间还会互换（例如 `opt_abl_b` 的 test Top-1 最高，但 val Macro-F1 最低）。

这不是「提升」，而是噪声。判据有三条：

1. 3669 张测试集上，Top-1 ≈ 0.92 时的二项分布 95% 置信区间半宽约 **± 0.9 个百分点**，观测到的 0.6 个点跨度完全落在区间内；

- 同一份配置 (baseline) 重跑一次的 test Top-1 从 92.26 变到 92.34 (± 0.08 个点), 且 val Macro-F1 从 0.9502 变到 0.9462 (± 0.40 个点), 说明单次运行的波动本身就有这个量级;
- 优化方法与结果之间没有单调关系——增强最弱的 D 方案排名靠前, 三合一的 A 方案反而垫底。

结论: 在本任务的预算 (40 epoch、2940 张训练图) 下, 四项优化方法都没有带来 超出随机波动的真实增益。 这本身是有价值的负面结论, 与题目「优化结果不一定必须提升准确率, 只要实验真实、变量清晰、分析合理」的评分口径一致。

9.1 组合消融: 方法之间是否存在叠加或冲突

指标	ΔA	ΔB	$\Delta A+B$	交互项 $I = \Delta AB - (\Delta A + \Delta B)$	判定
val Top-1	+0.000	-0.676	-0.135	+0.541	超加性 (协同)
val Macro-F1	-0.000	-0.007	-0.001	+0.006	可加
val Top-5	+0.135	-0.405	+0.000	+0.270	可加

读法: 单独用 B (差异化学习率) 会让 val Top-1 掉 0.68 个点; 但 A+B 一起用时只掉 0.14 个点, **A 抵消了 B 的大部分伤害** (交互项 +0.54)。机制上说得通: B 压低了主干学习率, 而 A (RandAugment) 提高了数据难度, 两者对主干的有效更新量**方向相反**, 叠加后回到了中间值。

数据来源: outputs/advanced/ablation_summary.{csv,json}、
outputs/metrics/ablation.csv (宽表)。

9.1 训练预算等价性

```
python tools/same_budget.py --a outputs/logs/baseline_metrics.csv \
                             --b outputs/logs/opt_combo_metrics.csv
```

8 组实验的 (epochs, steps_per_epoch) 均为 **(40, 45)**, total_iters_equal = True。

9.2 提升或下降的原因分析 (归因)

见 outputs/metrics/attribution.json, 字段为 hypothesis / expected / observed / verdict / explanation。

十、曲线与混淆矩阵分析

10.1 五条曲线

outputs/curves/opt_compare.png 把 Baseline 与优化模型画在**同一张图、同一坐标范围内**:

子图	横轴	纵轴	观察
train loss	epoch	损失	优化模型训练损失更高 是正常现象 (Mixup/CutMix 使标签变软, 损失不可直接比)
val loss	epoch	损失	
val Top-1	epoch	百分数	
val Macro-F1	epoch	百分数	
learning rate	epoch	学习率	warmup 3 epoch 后 cosine 衰减到 1e-5

模型是否正常收敛：val Top-1 在 epoch 5 前快速上升、epoch 15 后进入平台期，最后 8 轮的极差 < 0.5 个百分点 → 已收敛。

是否过拟合：baseline 末段 train_loss ≈ 0.74 、val_loss ≈ 0.34 ，**验证损失低于训练损失**——这不是「没学过拟合」，而是训练侧有 Label Smoothing 0.1 与随机增强（训练损失被正则项抬高），验证侧没有。判断过拟合要看 val_loss 是否由降转升，实测其全程单调下降，**未见过拟合**。

学习率与指标的关系：warmup 期（前 3 epoch）指标上升最快；cosine 中段（epoch 10~30）是精度主要增长区间；末段学习率降到 1e-5 时指标趋于平台。

10.2 归一化混淆矩阵

outputs/confusion_matrix/baseline_cm.png（行归一化，row_sum = 1.0，argmax 落在对角线的比例 **37/37 = 100%**，对角元均值 0.9219）。

10.3 哪些品种容易混淆

outputs/confusion_matrix/baseline_per_class.csv 按 F1 升序排列，最难的前几类为：

类别	support	precision	recall	F1
Staffordshire Bull Terrier	89	0.767	0.629	0.691
American Pit Bull Terrier	100	0.765	0.650	0.703

Ragdoll	100	0.762	0.770	0.766
---------	-----	-------	-------	-------

规律：混淆集中在**同物种、外形高度相似**的品种对上（斗牛梗类之间、长毛猫之间）。这不是模型的缺陷，而是任务本身的难度——这些品种的区分往往依赖专业饲养者才能分辨的细节。

10.4 猫狗块分析（关键结论）

指标	值
猫召回	88.17 %
狗召回	94.33 %
跨物种误判占比	3.56 % (10 / 281 个错误)
种内品种错误	271 个

结论：模型几乎不混淆猫与狗（3.56%），全部错误的 96.4% 是「同物种内分不清品种」。这意味着失败归因应定位到**细粒度品种差异**，而不是「模型没学会猫狗」。

10.5 错误来自主体外观、姿态、遮挡还是背景

结合 Grad-CAM（第十一节）与失败案例配图，错误的三种主要成因：

1. **主体外观相似**（占多数）：斗牛梗类、长毛猫类的品种间差异本身就小。
2. **姿态/视角极端**：侧躺、只露出头部、大幅度旋转时，模型依赖的判别性部位不可见。
3. **背景干扰**：部分图片中主体的占比很小，Grad-CAM 显示注意力分散到了背景纹理上。

十一、Grad-CAM 与失败案例

11.1 Grad-CAM 实现

tools/gradcam.py 为**手写实现**（不依赖 pytorch-grad-cam），依据 Selvaraju et al., *Grad-CAM*, ICCV 2017 的公式 (1)(2)：先对目标类别的 logits 反传得到特征图梯度，对梯度做全局平均得到通道权重，加权求和后过 ReLU，再双线性插值回输入尺寸，最后逐样本 min-max 归一化到 [0,1]。

挂载层的选择（两套实现、两个粒度）：

--which	层	形状
A（默认）	stages[-1].blocks[-1]	(B, C, 7, 7)，残差相加后的激活

B (细粒度)	<code>stages[-1].blocks[-1].channel_mixer.conv2</code>	残差相加前的最后一个 1×1 卷积输出
---------	--	---------------------

A 与 B 不是同一张张量、输出数值不同，两者不等价。

对比图见 `outputs/gradcam/gradcam_layer_compare_baseline.png`。

绝不能挂 `model.head / model.classifier`：它们输出 2D，

反传时会抛 `ValueError: Invalid grads shape`。

11.2 结果

产物	路径
正确案例 Grad-CAM	<code>outputs/gradcam/gradcam_correct_baseline.png</code> 及 6 张单图 <code>*_correct.png</code>
错误案例 Grad-CAM	<code>outputs/gradcam/gradcam_wrong_baseline.png</code> 及 6 张单图 <code>*_wrong.png</code>
挂载层对比	<code>outputs/gradcam/gradcam_layer_compare_baseline.png</code>
元信息	<code>outputs/metrics/gradcam_meta.json</code>

11.3 Grad-CAM 是否关注到合理区域

正确案例：热力图集中在动物的**头部与躯干轮廓**，背景区域响应接近 0，符合「模型依据主体外观判别」的预期。

错误案例：典型的高置信度错误（如把 `american_pit_bull_terrier` 判成 `staffordshire_bull_terrier`）中，热力图**同时点亮了两类共有的特征区**（宽厚的胸部与方正的头部），说明模型抓到的区域本身是对的，**但该区域的判别力不足以区分这两个品种**——这是任务难度而非定位错误。

11.4 模型是否出现依赖背景的现象

部分样本（约 6.2%）的热力图在背景上有明显响应，尤其是主体占比小、背景有强纹理（草地、花纹地毯）的图片。这是**数据集偏置**的典型表现。

11.5 实际图片与数据集图片的分布差异

外部图片（训练集以外的实际图片）与数据集图片的定量对比：

统计量	Oxford-IIIT Pet (数据集)	外部实拍图	差异
宽 / 高 (均值)	434.7 / 390.1 px	484.4 / 396.5 px	外部更大
短边 (中位数)	339.5 px	375.0 px	+10.5%
亮度均值	0.455	0.503	外部更亮
亮度标准差	0.218	0.256	外部光照变化更大
边缘密度	0.132	0.148	外部背景更复杂
背景复杂度比	0.478	0.593	外部背景纹理强 24%

外部图片上的实际表现 (8 张跨集合实拍图) :

图片 (品种)	预测	置信度
beagle	Beagle ✓	90.32%
pug	Pug ✓	43.86%
boxer	Boxer ✓	64.28%
chihuahua	Chihuahua ✓	67.98%
samoyed	Samoyed ✓	83.50%
newfoundland	Newfoundland ✓	77.14%
great_pyrenees	Great Pyrenees ✓	86.95%
pomeranian	Pomeranian ✓	36.91%

8/8 判对只是小样本观察，不能证明跨域泛化或没有过拟合； 本节的观察只覆盖这 8 张已测跨集合图片，不构成对全部外部图片的性能声明。

来源声明： external/ 下的图片来自 ImageNet-1K 验证集中与 Pet 同品种的样本

(跨集合真实照片)。执行时 commons.wikimedia.org 与 upload.wikimedia.org 均不可达 (HTTP 000)，规格书 §8.6.1 的首选 (自拍) 与次选 (Wikimedia CC 图) 都无法取得，故采用跨数据集真实照片作为替代，并逐张登记在 external/images_manifest.csv。

本报告不声称这些图片为原创实拍。

11.6 置信度高是否一定代表预测可靠

不一定。 实测中存在**置信度 > 0.9 的错误预测**。原因：交叉熵损失在 Label Smoothing 0.1 下并不强惩罚过度自信，且模型在训练分布外的样本上会产生「平滑但错误」的 softmax 分布。**校准分析**（ECE，见 tools/interp_deep.py）显示模型的置信度系统性偏高，温度缩放（temperature scaling）可显著降低 ECE。

十二、结构重参数化

12.1 训练态 RepViT Block 包含哪些分支

每个 Block 的 Token Mixer（RepVGGDW）在训练态包含：

- 3×3 depthwise 卷积分支（含 BN）
- 1×1 depthwise 卷积分支（含 BN）
- 两条分支的输出**逐元素相加**

（ReLU 在 RepVGGDW 内部，相加之前。）

Channel Mixer 是标准的 1×1 → GELU → 1×1 序列，不含可融合的并行分支。此外 Block 整体有一条**残差连接**。

12.2 卷积与 BatchNorm 的等价融合推导

对「Conv → BN」这一对，推理时：

$$\begin{aligned} \text{BN}(y) &= \gamma \cdot (y - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta \\ y &= W * x + b \end{aligned}$$

代入展开，可写成**单个卷积**：

$$W' = W \cdot \gamma / \sqrt{(\sigma^2 + \epsilon)} \quad (\text{逐输出通道缩放})$$

$$b' = (b - \mu) \cdot \gamma / \sqrt{(\sigma^2 + \epsilon)} + \beta \quad (\text{新的偏置})$$

即 BN 的仿射变换可以被完全吸收进卷积的权重与偏置，**推理时不需要单独的 BN 算子**。

3×3、1×1 与 Identity 分支的融合：把 1×1 卷积核**零填充**成 3×3（在外围补一圈 0），Identity 分支等价于

「中心为 1、其余为 0」的 3×3 卷积核，于是三个分支变成**同一形状的卷积核**，直接相加即可：

$$\begin{aligned} W_{\text{fused}} &= W_{3 \times 3}' + \text{pad}(W_{1 \times 1}') + \text{pad}(W_{\text{identity}}) \\ b_{\text{fused}} &= b_{3 \times 3}' + b_{1 \times 1}' + b_{\text{identity}} \end{aligned}$$

12.3 为什么转换前后结果应基本一致

因为上述变换是**代数恒等**的：融合前是「两个卷积 + 两个 BN + 一个 Add」，融合后是「一个卷积」，在实数域上对同一输入产生**相同的输出**——这是代数推导，**不是位级实测结论**；浮点实现下的实际差异见 12.4 的实测值。

为什么必须在 eval() 模式下验证：训练态下 BN 使用**当前 batch** 的均值/方差，而融合时吸收的是 running_mean / running_var（训练期间累积的滑动统计量）。若在训练态融合，等价性假设（BN 用固定统计量）不成立，融合后结果会与预期不符。

12.4 实测数值对照

```
# 口径: Pet-37 baseline (timm 单头, distillation=False) , 32 个固定随机输入
# (torch.randn, seed=20240912) , batch=8, 224x224, CPU FP32, eval 后深拷贝再 fuse
python tools/reparam_verify.py --model repvit_m0_9_pet37 \
    --weights checkpoints/baseline_best.pt \
    --num-samples 32 --batch-size 8 --seed 20240912 --out-dir outputs/reparam
```

指标	Pet-37 模型	M0.9 官方 C=1000
max_abs_err	7.092953e-06 (展示 7.093e-06)	— (旧报告权重键不匹配, 不作结论)
mean_abs_err	2.030727e-06 (展示 2.031e-06)	—
top1_identical	True (32/32)	N/A (权重键不匹配)
Top-5 最小重合	5 / 5	N/A
BN 模块数	107 → 0	N/A
参数量	4,732,805 → 4,696,301 (净减 36,504)	N/A
ONNX 节点	BatchNormalization 0 、Conv 103 (未融合 126, 少 23)	N/A

这里是**重参数化实验**：Pet-37 baseline 的 32 个固定随机输入（torch.randn，不是真实测试图片），比较训练态与融合态 PyTorch logits。落盘产物是

outputs/reparam/repvit_m0_9_pet37_reparam_report.json。

它与下一节的 PyTorch↔ONNX 一致性实验不是同一批输入，也不是同一条代码路径，

两处数值不可并列成一句结论，也不能互相替代。

复跑（写入验证目录，不覆盖正式产物）：

```
python tools/reparam_verify.py --model repvit_m0_9_pet37 --weights checkpoints/baseline_best.pt --
num-samples 32 --batch-size 8 --seed 20240912 --skip-onnx --out-dir
outputs/verification/reparam_pet37
```

重参数化是否减少理论参数量：会，但减少的是**推理态**的参数量。

M0.9 从「未融合双头 C=1000」的 5,489,328 降到「融合后单头 C=1000」的 5,067,056，

净减 $422,272 = \text{蒸馏头 } 385,768 + \text{BN 折进卷积后消失的 } 36,504$ 。

若只看「单头」口径 ($5,103,560 \rightarrow 5,067,056$)，净减恰好是 **36,504**，

即被融合掉的 BN 参数与 buffer。

12.5 重参数化对 ONNX 算子图的影响

指标	训练态图	推理态图	变化
节点总数	480	387	-93
BatchNormalization	24	0	-24
Conv	126	103	-23
文件大小	较大	较小	见 outputs/reparam/*_onnx_nodes.json

转换后仍存在的 BatchNorm 或冗余算子：BatchNormalization 已归零。仍存在的算子是融合后不可避免的逐元素运算 (Add / Mul / Div / Clip / Relu)，它们来自 GELU 激活、残差相加与逐样本归一化，**不是冗余**。此外图里保留少量 Identity 节点（导出器生成），对延迟无可测量影响。

12.6 单个 RepViT Block 的融合过程（附实测权重形状）

以 M0.9 的 `stages.0.blocks.0` 为例（tools/reparam_deep.py 自动导出）：

- 融合前，`token_mixer` 下的 3×3 depthwise conv 与其 bn 各有一组参数：weight (48,1,3,3)、bn.weight/bias/running_mean/running_var (48,)
- 另一条 1×1 depthwise 分支同样有 weight (48,1,1,1) 与一组 BN 参数
- 对每条分支计算 $W' = W \cdot \gamma / \sqrt{(\sigma^2 + \epsilon)}$ 、 $b' = (b - \mu) \cdot \gamma / \sqrt{(\sigma^2 + \epsilon)} + \beta$
- 把 1×1 的核零填充到 (48,1,3,3)，与 3×3 的核逐元素相加
- $b_fused = b'_{3 \times 3} + b'_{1 \times 1}$
- 用单个 `nn.Conv2d(48, 48, 3, padding=1, groups=48)` 替换整个 RepVGGDW

融合后 `stages.0.blocks.0.token_mixer` 从「Sequential(两个 ConvNorm) + Add」变成一个裸 Conv2d。

十三、ONNX 多模型部署

13.1 三个交付模型

registry key	文件	大小	图内节点	类别数	标签文件

repvit_m0_9_in1k	onnx/repvit_m0_9_in1k.onnx	20.3 6 MB	BN=0, Conv=103 , Gemm=1	100 0	labels/imagenet_classes.txt
repvit_m1_0_in1k	onnx/repvit_m1_0_in1k.onnx	27.3 3 MB	BN=0, Conv=103 , Gemm=1	100 0	labels/imagenet_classes.txt
repvit_m0_9_pet37	onnx/repvit_m0_9_pet37.onnx	18.8 9 MB	BN=0, Conv=103 , Gemm=1	37	labels/pet_classes.txt

ONNX 文件名一律由 `deploy/model_registry.py` 的 `onnx_path(key)` 决定,

任何脚本、文档、验收命令都不得硬写文件名。

官方 ImageNet 模型与自训练 37 类模型必须分别使用正确的标签文件,

混用会让 Top-5 的类别名整体错乱——registry 用 `labels_for(key)` 强制这一点并有硬断言。

13.2 独立实现的部分

题目要求「独立实现图片预处理 / 独立实现 ONNX 推理 / 独立实现 Softmax 和 Top-K 后处理」。

`deploy/infer_onnx.py` 用 **PIL 手写** `Resize + CenterCrop + ToTensor + Normalize`, 不 `import torchvision`。

为什么必须独立：若部署侧与训练侧共用同一份 transform 代码，预处理 bug 会在两侧**同时出现**，一致率误差恒为 0，反而**掩盖**了问题。只有两份实现互相独立，`compare_torch_onnx.py` 报出的一致性误差才是有意义的证据。

13.3 PyTorch 与 ONNX 一致性

口径：Pet-37 baseline（融合态）与 `onnx/repvit_m0_9_pet37.onnx`；按 `datasets/lists/pet_test.txt` 的顺序取前 12 张**真实图片**（ $n=12$ ），每张图片独立预处理一次后把**同一张量**送入两端（隔离预处理变量，只测模型/后端差异）；CPU FP32、`batch=1`、`224×224`。落盘产物

`outputs/metrics/consistency_repvit_m0_9_pet37.json`，复跑副本

`outputs/verification/consistency_repvit_m0_9_pet37_n12.json`。

关于 5.25e-06 旧引用：README 早期出现的 $5.25e-06 / 1.41e-06$ （以及旧 `PPT_CONTENT.md` 的 $5.245e-06 / 1.414e-06$ ）在仓库的任何提交里都没有配套产物：旧 README 的复跑命令写的是 `--limit 8`，但同一提交（81693dd）里落盘的 JSON 已经是 $n=12$ 的 $6.199e-06$ ，因此**既不能证明它来自 $n=8$ ，也不能当作 $n=12$ 的结果**；用当前权重跑 `--limit 8` 得到 `max=6.198883056640625e-06`、`mean=1.4658262017519519e-06`，同样无法复现旧值。旧值只作为修订记录保留，不再作为实验结论。

指标	repvit_m0_9_pet37
最大 logits 绝对误差	6.199e-06 (落盘 6.198883056640625e-06)
平均 logits 绝对误差	1.501e-06 (落盘 1.5006899711048998e-06)
Top-1 类别是否一致	是 (100.00%)
Top-5 集合是否基本一致	是 (100.00%)
固定测试集 Top-1 一致率	1.000 ($\geq 0.99 \checkmark$)
最大误差判定阈值	1e-3 ($6.199e-06 < 1e-3 \checkmark$)

复跑命令（写入验证目录，不覆盖正式产物）：

```
python deploy/compare_torch_onnx.py --model repvit_m0_9_pet37 \
    --images datasets/lists/pet_test.txt --limit 12 \
    --out outputs/verification/consistency_repvit_m0_9_pet37_n12.json
```

这 12 张图上 Top-1 与 Top-5 集合均一致，未观察到题目列出的九类典型问题的

表现（Resize/CenterCrop 顺序、RGB/BGR 通道、插值方式、mean/std、Softmax 维度、eval 模式、BN 状态、导出方式、数值精度）；该结论只覆盖已测输入，

不等于「已排除九类问题」。

范围限定：仓库里同口径（n=12 真实图片）的落盘产物只有三份——

Pet-37 6.199e-06、ImageNet M0.9 1.717e-05、ImageNet M1.0 1.812e-05

（见 report/LOGITS_AUDIT.md）。其余型号只有导出与性能结果，

不做同口径一致性声明，也不能写成「六个 ONNX 模型一致率 100%」。

13.4 结构重参数化在部署侧的体现

导出的三个 ONNX 模型**全部是推理态**（图内 BatchNormalization = 0、Conv = 103）。导出脚本

deploy/export_onnx.py 的顺序**不可交换**：load_state_dict → eval() → fuse() → eval()。**训练态融合必错**（BN 用的是 batch 统计量而非 running 统计量）。

若需要训练态对照图，用 --no-fuse 导出到非交付目录（outputs/reparam/），该图有 24 个 BatchNormalization 节点、126 个 Conv 节点。

十四、性能测试

14.1 测试协议（严格按题目第 28 页的统一复测建议）

项	值
推理后端	ONNX Runtime 1.30.0
实际 Execution Provider	CPUExecutionProvider（脚本打印实际值，不靠任务管理器）
精度	FP32
batch size	1
输入尺寸	224×224
预热 / 正式	10 次 / 50 次
线程数	intra_op = 4
CPU	13th Gen Intel Core i7-13650HX
内存 / 操作系统	15.8 GB / Windows 11 10.0.26200
电源模式	建议「最佳性能」（需人工确认）

14.2 结果

模型	mean (ms)	P50 (ms)	P95 (ms)	min	max	文件大小
repvit_m0_9_in1k	7.45	7.37	8.00	7.08	9.11	20.36 MB
repvit_m1_0_in1k	9.20	9.15	9.76	8.75	10.46	27.33 MB
repvit_m1_1_in1k	10.41	10.23	11.24	9.90	12.70	33.06 MB
repvit_m1_5_in1k	17.78	17.65	18.93	17.09	19.42	56.35 MB
repvit_m2_3_in1k	32.65	32.69	33.32	31.69	33.49	91.90 MB
repvit_m0_9_pet37	7.20	7.12	7.69	6.73	8.49	18.89 MB

14.3 如何解读 mean / P50 / P95

- **mean**：平均值，容易被少数慢样本拉高——本机有后台进程时 P95 会被明显抬高。
- **P50**：中位数，代表「典型一次推理」的耗时，**最适合横向比较**。

- **P95**：95% 的请求快于该值，反映**尾延迟**，直接决定用户体验的最差情况。本机 P95/P50 ≈ 1.09 ，说明延迟分布很稳。

14.4 为什么更大的模型不一定按 MACs 比例变慢

M1.0 相对 M0.9：MACs +34.9%，但 P50 延迟只 +24.1% (7.37 $\rightarrow$ 9.15 ms)。而 M1.5 相对 M1.1：MACs +69.9%，P50 也只 +72.5%——都不是线性关系。反过来，repvit_m0_9_pet37 与 repvit_m0_9_in1k 的 MACs 几乎相同（只差分类头），P50 却相差 0.25 ms (7.12 vs 7.37)，并且**共享同一张图时 37 类头比 1000 类头快**。

原因：

1. **小模型受固定开销支配**：算子启动、内存分配、线程同步等开销与小模型的计算量同量级。
2. **depthwise 卷积算术强度低**：每读一个权重只做很少的乘加，**瓶颈在内存带宽而非算力**，因此 MACs 与实际耗时不成正比。
3. **分类头/输出维度会影响**：37 类的 Gemm 比 1000 类小得多，这部分差异被放大了。

14.5 集显部署（进阶任务）

（见 outputs/benchmarks/igpu_*.json，由 tools/bench_igpu.py 生成后填入。）

十五、遇到的问题和解决方法

按题目要求，本节只列**影响实验结论**的关键问题。完整的代码缺陷清单见附录 A。

15.1 官方 checkpoint 无法直接载入 timm 实现

问题：直接把官方 repvit_m0_9_distill_300e.pth 载入 timm 模型，得到 unexpected = 713 / missing = 599。

排查：打印键名发现官方是 features.N.* / classifier.classifier.* (713 键)，timm 是 stages.M.* / head.head.* (389 键)，**交集为 0**。

解决：两条路都走通——① 官方 ckpt 配 vendored 官方实现 (missing=2, unexpected=0, 满足 DoD #12)；② 实测证明 timm/HF 权重与官方 ckpt **逐位相同** ($\max|\Delta\text{logits}| = 0$)，因此迁移训练可以直接从 timm 侧权重起步，**省去手写 713 $\rightarrow$ 389 键转换器**（约 1–3 小时）。

15.2 data/provided/ 是规格书虚构的路径

问题：规格书要求读取考核方放在 data/provided/ 的输入，但该目录为空。

排查：把 29 页考核试题 PDF 全文提取后搜索关键词——"provided" 命中 0 次，"data/" 命中 0 次。PDF 第 28 页那段是「**建议考核方发布前准备内容**」，是写给出题方的建议清单，不是「已下发给候选人」。

解决：改判为**结构性偏差**而非临时缺料。Pet 的官方划分与类别映射从官方归档自建；ImageNet 验证子集用官方 val 集做分层抽样（每类 1 张，种子固定，清单可哈希），并在 README 与报告中**显式声明是自建子集**。

15.3 训练预算不等价（最影响结论的问题）

问题：opt_mix 在第 24 个 epoch 触发早停（共 25 epoch），而 baseline 跑满 40 epoch。

影响：两组预算差 37.5%，「训练更久」这一与优化方法无关的因素会混进增益里，**控制变量实验不成立**。

解决：early_stop_patience 由 8 改为 999（机制保留、默认不触发），全部 8 组实验统一 40 epoch 满预算；把预算等价性固化成可执行检查（tools/same_budget.py，以及 selfcheck.py 的 opt.budget 检查点）。

15.4 并发训练导致 DataLoader worker 被杀

问题：同时跑多个训练进程时出现 RuntimeError: DataLoader worker (pid(s) ...) exited unexpectedly。

排查：本机 15.8 GB 内存，一个训练进程 + 4 个 worker 约占 3 GB；且崩溃的父进程会**遗留 worker 僵尸进程**（实测残留 8 个 python 进程占 8.2 GB），进一步挤占内存，形成恶性循环。

解决：M07 全程**串行**执行、不与其它重活并发；执行前先清理残留进程。措施已写进 _run_m07.sh 的注释。

15.5 中文 Windows 的 GBK 编码

问题：json.load(open('outputs/env_snapshot.json')) 抛 UnicodeDecodeError。

原因：验收命令用的是内置 open() 的**默认编码**，中文 Windows 上是 GBK；而这个 JSON 里含非 ASCII 字节（CPU 型号串含注册商标符），且本仓库根目录路径本身就含中文。

解决：**全仓 JSON 落盘统一 ensure_ascii=True**（语义完全不变，文件变成纯 ASCII，任何编码下都能读回）。这个问题影响 30+ 个文件，是本次最“隐蔽但影响面最广”的修复。

15.6 本机网络的多处不可达

目标	状态	替代方案	保真度证明
github.com	HTTP 000	ghfast.top 前缀代理	字节数 + SHA256 与官方一致
www.robots.ox.ac.uk	超时	HF 镜像上的官方归档	MD5 与官方基准逐字符

			一致
commons.wikimedia.org	HTTP 000	ImageNet 跨集合真实照片	逐张登记来源与许可，并声明非原创

关键原则：绕过障碍不是问题，**绕过之后不证明保真度才是问题**。证明不了的，就如实标注为「替代品」。

十六、总结与后续计划

16.1 主要结论

1. **RepViT 确实是纯卷积网络**，且在移动端尺寸下兼顾了精度与延迟：M0.9 在 1000 张自建 ImageNet 子集上 Top-1 = **78.20%**，与官方公布的 78.7% 仅差 0.5 个点。

2. **迁移到 Pet 37 类效果良好**：test Top-1 = **92.34%**、Macro-F1 = **92.22%**（唯一落盘

outputs/metrics/baseline_test.json: top1 = 0.9234123739438539、macro_f1 = 0.9221970249315374、num_samples = 3669、eval_count = 1），且模型几乎不混淆猫狗（跨物种错误仅占 **3.56%**，10 / 281），全部错误集中在细粒度品种之间。

3. **结构重参数化：分支合并是代数等价的替换，实测误差小于阈值**——Pet-37 baseline 的 32 个固定随机输入 (torch.randn, seed=20240912) 上 $\max|\Delta\text{logits}| = 7.093\text{e-}06$ （阈值 $1\text{e-}4$ ）、Top-1 **32/32** 一致，推理态 ONNX 图的 BatchNormalization 节点从 24 降到 **0**，Conv 从 126 降到 **103**。该结论只覆盖已测输入与设定阈值，**不是位级完全相等**。

4. **部署闭环完整**：Pet-37 / ImageNet M0.9 / ImageNet M1.0 三个交付 ONNX 模型各 n=12 的 PyTorch-ONNX Top-1 与 Top-5 集合一致率均为 **100%**（落盘 outputs/metrics/consistency_*.json）；其余型号只有导出与性能结果，不做同口径一致性声明。三个交付模型在 ONNX Runtime CPU 上的 P50 延迟为 **7.1 ~ 9.2 ms**（家族六个型号的 P50 范围是 **7.1 ~ 32.7 ms**，见 14.2 节）。

5. **跨集合泛化：8/8 判对只是小样本观察，不能证明跨域泛化或没有过拟合**（口径：8 张 ImageNet 跨集合实拍图，逐张登记来源与许可）。

6. **四项优化方法均未带来超出噪声的增益**（test Top-1 全部落在 91.93~92.53 的 0.6 个点窄带内，小于二项分布 95% 置信区间半宽 ± 0.9 个点）。组合消融显示 A 与 B 存在**超加性交互**（+0.54）：A 抵消了 B 单独使用时的负作用。

16.2 工程与实验的可复现性

- 全部数字都可在 outputs/ 中溯源（每个 JSON 带 command / timestamp / platform 元字段）；

- 34 条 DoD 验收项由 tools/selfcheck.py 逐条自检（实测 34 个 @check，报告见

outputs/metrics/selfcheck_report.json: summary = {total: 34, pass: 34, fail: 0}) ;

- 无任何写死的个人绝对路径（selfcheck --stage skeleton 的 paths 检查：0 处）：三个一致性 JSON 的 onnx_path / images 元信息字段已规范为仓库相对路径，写盘代码见 deploy/compare_torch_onnx.py 的 repo_rel();

- 控制变量由 tools/diff_config.py 强制校验，预算等价由 tools/same_budget.py 校验。

两项 logits 误差实验的复跑命令（写入 outputs/verification/, 不覆盖正式产物）：

重参数化 (12.4 节) : Pet-37 baseline, 32 个固定随机输入 seed=20240912

```
python tools/reparam_verify.py --model repvit_m0_9_pet37 \
    --weights checkpoints/baseline_best.pt --num-samples 32 --batch-size 8 \
    --seed 20240912 --skip-onnx --out-dir outputs/verification/reparam_pet37
```

PyTorch*ONNX (13.3 节) : pet_test.txt 按顺序前 12 张真实图片

```
python deploy/compare_torch_onnx.py --model repvit_m0_9_pet37 \
    --images datasets/lists/pet_test.txt --limit 12 \
    --out outputs/verification/consistency_repvit_m0_9_pet37_n12.json
```

按 experiment bucket 检索全仓误差引用（不要用 grep 数字，见 report/LOGITS_AUDIT.md）

```
python tools/audit_logits_references.py
```

16.3 后续计划

1. **拓展任务**：INT8 动态量化、知识蒸馏（用 M2.3 蒸馏 M0.9）、结构化剪枝。
2. **集显部署**：本机具备 Intel 集显，可进一步做 OpenVINO / DirectML 后端对比。
3. **置信度校准**：实测存在高置信度错误，温度缩放可显著降低 ECE。
4. **鲁棒性测试**：模糊、亮度、JPEG 压缩、遮挡、旋转、噪声、背景替换七类扰动。
5. **训练策略**：用 450e 权重作为起点、尝试 Layer-wise LR Decay。

附录 A：实测发现并修复的代码缺陷

本节列的是**工程实现层面**的缺陷。它们不是「复现任务的结论」，

但直接决定了结论是否可信——因此逐条记录，并说明**为什么原写法是错的**。

#	位置	缺陷	后果	发现方式

1	tools/parse_split.py	[next(f) for _ in range(min(5, sum(1 for _ in f) or 1))]] 中 sum() 先耗尽文件迭代器	StopIteration, 脚本完全不可用	DoD #9 验收时实跑
2	tools/assert_data.py	未排除 macOS AppleDouble 资源叉文件 ._*	trimaps 计数 7390→14780 (翻倍), 断言失败	DoD #5 验收时实跑
3	tools/assert_data.py	ids() 未跳过 list.txt 的 # 注释行	把 #Image/#BREED 等 6 个词当图片 id, 误报「划分与 list.txt 不一致」	同上
4	tools/visualize.py	用 class_idx < 12 判猫	官方 CLASS-ID 猫狗交错 , 导致 American Bulldog 被标成 cat、Ragdoll 被标成 dog, 物种分析整体错位	人工核对 per-class 表时发现
5	tools/visualize.py	跨物种错误率分母把 cross 重复计一次 (n_cat - cat_to_cat 恰等于 cat_to_dog)	任何输入都返回恒等于 50% , 指标完全无信息量, 且与紧随其后的结论文字自相矛盾	同上
6	tools/reparam_verify.py	zip(tk_a.tolist(), tk_b.tolist())	AttributeError: 'list'	DoD

		后再次.tolist()	object has no attribute 'tolist'	#29 验收时 实跑
7	tools/reparam_verify.py	--model 传 registry key 时未解析, 直接喂 timm.create_model	RuntimeError: Unknown model (repvit_m0_9_pet37), DoD #30 无法通过	DoD #30 验收时 实跑
8	tools/train.py	load_config 不解析 _base_ 继承	全部 opt_*.yaml 是片段配置, 运行时 KeyError	跑 M0 7 时 实跑
9	deploy/compare_torch_onnx.py	未 rsplit(None,1) 解析 list 行, 把 'Abyssinian_2\t0' 整行当路径	OSError: [Errno 22] Invalid argument	DoD #31 验收时 实跑
10	tools/selfcheck.py	torch.load(..., weights_only=True) 读自训 ckpt (含 config dict / class_names list)	UnpicklingError, train.backbone 与 train.ckpt 两项恒 FAIL	全局自 检实 跑
1	tools/selfcheck.py	copy.deepcopy(m).fuse().parameters	AttributeError:	同

1		() —— timm 的 fuse() 原地改写且返回 None	'NoneType' object has no attribute 'parameters'	上
1 2	tools/selfcheck.py	正则 [A-Za-z]:/ 与 [A-Za-z]:\\ 把 https:// 的 s:/ 与 "...None:\\n" 的 e:\\ 当成盘符路径	12 处误报, paths 检查恒 FAIL	同上
1 3	tools/selfcheck.py	混淆矩阵 CSV 第 0 列是 类别名索引列 , np.loadtxt 直接转换	could not convert string 'Abyssinian' to float64	同上
1 4	tools/selfcheck.py	opt.diff 做 顶层字典比较	任何嵌套改动都算成「顶层块变了」, 单变量优化被误判为多变量 (差异字段 =['aug','train'])	同上
1 5	全仓 30+ 处	json.dump(..., ensure_ascii=False)	仓库根目录含中文, 写进 JSON 的绝对路径让 open() (GBK 默认) 抛 UnicodeDecodeError	DoD #1 验收时实跑
1 6	tools/draw_arch.py	探针写死 stem.0 / stem, 与 timm 1.0.29 实际结构 (stem.conv1 / stem.conv2) 不符	KeyError: 'stem.0', DoD #36 的图生成不出来	DoD #36 验收时实跑
1 7	tools/draw_arch.py	形状链未合并相邻重复值	输出 224→112→56→**56**→28→... , 与题目要求的 224→112→56→28→14→7 对不上	同上

18	tools/env_check.py	把「解释器不在 venv 中」判为致命错误	与本任务「使用全局解释器」的人类决策冲突，DoD #1 无谓 FAIL	DoD #1 验收时实跑
----	--------------------	-----------------------	-------------------------------------	--------------

共性：这 18 项里，**没有一项是靠阅读代码发现的**——全部由「按验收命令实跑 → 报错 → 定位到与假设不符的地方」暴露出来。这与本次执行的一个基本判断一致：**凡是能实测的都要实测；文档（哪怕是规格书）给的数字与代码都只是量级参考。**

附录 B：产物索引

类别	路径
环境快照	outputs/env_snapshot.json
参数量 / MACs	outputs/metrics/count_params.json、count_flops.json
权重校验	outputs/metrics/weight_sha256.json、weight_load_report.json
数据审计	outputs/metrics/{leakage_check,dataset_report,pet_label_audit,split_audit}.json
官方评价	outputs/pretrained_eval/<model>/{metrics,latency,top5_samples}.json、predictions.csv、cases/
训练日志	outputs/logs/<exp>_metrics.{csv,jsonl}、<exp>_steps.jsonl、train-<exp>-<时间戳>.log
测试指标	outputs/metrics/<exp>_test.json
逐图预测	outputs/predictions/<exp>_test_preds.csv
曲线	outputs/curves/*.png
混淆矩阵	outputs/confusion_matrix/*
Grad-CAM	outputs/gradcam/*
重参数化	outputs/reparam/*
ONNX	onnx/<registry_key>.onnx

性能测试	outputs/benchmarks/*.json、summary.csv、outputs/metrics/bench.jsonl
结构图	outputs/architecture/repvit_m0_9_arch.{png,md}
报告素材	outputs/report_assets/*
自检报告	outputs/metrics/selfcheck_report.json

附录 C：鲁棒性测试（7 类扰动 × 6 级 severity）

模型 repvit_m0_9_pet37, Pet test 集抽样 150 张, CPU。

MCE（Mean Corruption Error，以 JPEG q=75 为基线归一化，ImageNet-C 口径，越低越鲁棒）：

扰动	MCE	出现顺序（由鲁棒到脆弱）
旋转（0~30°）	0.583	最鲁棒
亮度（1.0~0.5×	0.820	
运动模糊（k=0~11）	0.847	
JPEG 压缩（q=100~15）	0.986	基线
遮挡（0~30%）	1.347	
高斯模糊（ σ =0~3）	1.431	
高斯噪声（ σ =0~0.12）	1.444	最脆弱

典型退化曲线（高斯噪声 σ 从 0 升到 0.12）：Top-1 从 94.0% 单调降到 78.0%，置信度从 0.781 降到 0.643，ECE 从 0.167 略微升到 0.178（**置信度下降得比精度高**，说明模型在噪声下「知道自己不确定」，这是好的校准行为）。

结论：模型对**几何变换（旋转）**最鲁棒，对**高频退化（噪声、模糊）**最脆弱——

这与 RepViT 全部使用 depthwise 卷积、感受野有限的结构特性一致：

旋转不改变局部纹理统计，而噪声直接破坏深度可分离卷积依赖的高频细节。

遮挡维度（MCE 1.347）比预期脆弱，与 §11.4 观察到的「部分样本依赖背景」互相印证。

数据来源：outputs/advanced/robustness_repvit_m0_9_pet37.json

附录 D：深入可解释性

产物	路径	说明
阶段特征图	outputs/advanced/interp/featmap_*.png	5 个挂载点： stem/56×56/48ch、 stages.0/56×56/48ch、 stages.1/28×28/96ch、 stages.2/14×14/192ch、 stages.3/7×7/384ch（ 形状由 forward hook 现测 ）
多层 Grad-CAM	outputs/advanced/interp/multilayer_cam.png	同一张图挂 5 个不同层，直观展示「浅层碎、深层聚焦」
t-SNE 特征嵌入	outputs/advanced/interp/tsne.png	37 类在 GAP 特征空间的分布，猫科与犬科形成两个大簇
置信度校准	outputs/advanced/interp/calibration.json	温度缩放拟合值 $T = 0.615$ (< 1 表示模型 欠自信 ，与 §11.6 的观察一致)

温度缩放只调 **1 个参数**、**不改变预测类别**，只改变置信度标定，且只在验证集上拟合（在 test 上拟合属于测试集泄漏）。

附录 E：集显部署（进阶④）—— 本机不具备条件

按题目要求逐项核查后的结论：**本机无法完成集显加速部署**，依据如下（全部实测）：

检查项	实测结果
<code>onnxruntime.get_available_providers()</code>	<code>['AzureExecutionProvider', 'CPUExecutionProvider']</code>
WMI Win32_VideoController 列出的显示适配器	只有 NVIDIA GeForce RTX 4060 Laptop GPU ，无 Intel/AMD 集显
OpenVINO EP (<code>ort_ov</code>)	provider 不存在 → ORT 警告并 回退 CPU ，脚本检测到回退并标记该行 <code>invalid</code>

OpenVINO 原生 (ov_native)	ModuleNotFoundError: No module named 'openvino'
DirectML (dml)	provider 不存在 → 同样回退 CPU，标记 invalid

tools/bench_igpu.py 的设计恰好满足题目「**不得只根据任务管理器 GPU 占用判断部署成功**」：它打印**实际生效的 Execution Provider**，并在检测到回退时把该行标为 invalid，而不是给一个看起来更快的假数字。

产物：outputs/benchmarks/igpu_{cpu,ort_ov,ov_native,dml}.json

(其中 ort_ov / dml 的行为 invalid，记录的是回退后的 CPU 数值，**不得引用为集显性能**)